Computability - Topic 3

Yonghao Lee

February 2026

1 Context-Free Languages

We have seen two equivalent methods of describing languages: finite automata and regular expressions. While many languages can be described this way, we proved that some cannot (e.g., $\{0^n 1^n \mid n \geq 0\}$).

We now introduce **context-free grammars** (CFGs), a more powerful method of describing languages. CFGs are more powerful because they can describe languages that require matching or counting — features that have a recursive structure. This makes them useful in many fields, particularly in programming language design and parsing.

The collection of languages associated with context-free grammars are called the **context-free languages** (CFLs). They include all the regular languages and many additional ones.

1.1 Informal Description

A grammar consists of a collection of **substitution rules**, also called **productions**. Each rule has a **variable** (also called a nonterminal) on the left side and a string of variables and **terminals** on the right side, separated by an arrow. One variable is designated as the **start variable**.

We use a grammar to describe a language by *generating* strings as follows:

1. Write down the start variable.
2. Find a variable in the current string and a rule whose left side matches that variable. Replace the variable with the right side of the rule.
3. Repeat step 2 until no variables remain.

All strings generated in this way constitute the language of the grammar.

1.2 Formal Definition

Definition 1.1: Context-Free Grammar

A **context-free grammar** (CFG) is a 4-tuple (V, Σ, R, S) where:

- V is a finite set called the **variables** (nonterminals)
- Σ is a finite set, disjoint from V, called the **terminals**
- R is a finite set of **rules**, where each rule has the form $A \to w$ with $A \in V$ and $w \in (V \cup \Sigma)^*$
- $S \in V$ is the **start variable**

1.3 Designing Context-Free Grammars

We now describe several useful strategies for constructing CFGs.

Strategy 1: Union of Simpler Languages

Many CFLs are the union of simpler CFLs. If we can decompose a CFL into simpler pieces, we construct individual grammars for each piece and then merge them: combine all rules and add a new start rule $S \to S_1 \mid S_2 \mid \cdots \mid S_k$, where S_i is the start variable of the i-th grammar.

Example 1.1: Union Construction

Consider the language $\{0^n1^n \mid n \geq 0\} \cup \{1^n0^n \mid n \geq 0\}$. We first construct grammars for each part:

$$S_1 \to 0S_11 \mid \varepsilon \qquad\qquad S_2 \to 1S_20 \mid \varepsilon$$

Then we merge them with the rule $S \to S_1 \mid S_2$.

Strategy 2: Converting a Regular Language

If the language happens to be regular, we can first construct a DFA for it and then convert the DFA into a CFG mechanically: create a variable R_i for each state q_i, and for each transition $\delta(q_i, a) = q_j$, add the rule $R_i \to aR_j$. For each accept state q_j, add $R_j \to \varepsilon$. The start variable corresponds to the start state.

Strategy 3: Linked Substrings

Certain CFLs contain strings with two substrings that are "linked" — a machine would need to remember an unbounded amount of information about one substring to verify that it corresponds properly to the other. Our classic example is $\{0^n1^n \mid n \geq 0\}$, where the number of 0s must match the number of 1s.

To handle this, we use a rule of the form $R \to uRv$, which generates matching characters u and v on either side of R simultaneously. Each application of this rule adds one u on the left and one v on the right, ensuring they stay balanced. For example, $S \to 0S1 \mid \varepsilon$ generates 0^n1^n because each application adds a matched pair.

Strategy 4: Recursive Structure

In more complicated languages, strings may contain structures that appear recursively as part of other (or the same) structures. For example, arithmetic expressions like $((3+5)\times 2)$ have nested parenthesized subexpressions.

To achieve this, place the variable that generates the structure at the location in the rule where the structure may recursively appear. For instance, a grammar for simple arithmetic expressions might include:

$$E \to E + E \mid E \times E \mid (E) \mid a$$

Here, the variable E appears inside the rules for E itself, allowing expressions to nest arbitrarily deep.

1.4 The Pumping Lemma for Context-Free Languages

Just as the Pumping Lemma for regular languages allowed us to prove that certain languages are not regular, there is an analogous result for context-free languages. The idea is similar: sufficiently long strings in a CFL must contain a section that can be "pumped." However, the structure is slightly more complex — instead of one pumpable piece, we get two pieces that must be pumped together.

Theorem 1.1: Pumping Lemma for Context-Free Languages

If A is a context-free language, then there exists a number p (the pumping length) such that if s is any string in A with $|s| \geq p$, then s may be divided into five pieces, $s = uvxyz$, satisfying the following conditions:

1. $uv^ixy^iz \in A$ for every $i \geq 0$
2. $|vy| > 0$
3. $|vxy| \leq p$

When s is divided into $uvxyz$, both v and y are the pieces that can be pumped. Condition 2 says that at least one of v and y is nonempty. Condition 3 states that the middle portion vxy has length at most p, which constrains where the pumpable sections can appear. Note that v and y must be pumped *simultaneously* and by the same power i: you cannot pump one without the other.

The proof relies on parse trees rather than automata. If a string is long enough, its parse tree must be tall enough that some variable repeats along a path from the root to a leaf. The subtree rooted at

the higher occurrence of the repeated variable generates the portion vxy, and the subtree at the lower occurrence generates x. Replacing one subtree with the other allows us to pump v and y simultaneously.

Using the Pumping Lemma for CFLs

The usage is analogous to the regular language version:

1. Assume for contradiction that B is context-free.
2. The Pumping Lemma guarantees a pumping length p.
3. Choose a specific string $s \in B$ with $|s| \geq p$.
4. Show that *no* division $s = uvxyz$ satisfying conditions 2 and 3 allows condition 1 to hold, by finding a value of i such that $uv^ixy^iz \notin B$ for every valid decomposition.

1.5 Example: $\{a^nb^nc^n\}$

We show that $B = \{a^nb^nc^n \mid n \geq 0\}$ is not context-free.

Proof. Assume for contradiction that B is context-free. Let p be the pumping length given by the Pumping Lemma for CFLs. Choose $s = a^pb^pc^p$. Since $s \in B$ and $|s| = 3p \geq p$, the Pumping Lemma guarantees that s can be divided into five pieces $s = uvxyz$ satisfying the three conditions.

By condition 3, $|vxy| \leq p$. Since $s = a^pb^pc^p$ and vxy is a substring of length at most p, the portion vxy cannot span all three blocks of characters. That is, vxy can touch at most two of the three symbols a, b, c.

By condition 2, at least one of v and y is nonempty. So pumping increases the count of at most two of the three symbols. Consider $i = 2$:

$$uv^2xy^2z$$

This string has more of at most two symbols, while the third remains at p. Therefore the three counts are no longer equal, so $uv^2xy^2z \notin B$. This contradicts condition 1.

Therefore B is not context-free. □

1.6 Example: $\{ww\}$ Revisited

We show that $D = \{ww \mid w \in \{0,1\}^*\}$ is not context-free.

Proof. Assume for contradiction that D is context-free. Let p be the pumping length given by the Pumping Lemma for CFLs. Choose $s = 0^p1^p0^p1^p$. Since $s \in D$ (with $w = 0^p1^p$) and $|s| = 4p \geq p$, the Pumping Lemma guarantees a decomposition $s = uvxyz$ satisfying the three conditions.

By condition 3, $|vxy| \leq p$, so vxy is contained within a substring of length p. In the string $0^p1^p0^p1^p$, this means vxy can overlap at most two of the four blocks. We consider cases based on where vxy falls:

- vxy **lies within the first** 0^p**:** then pumping changes only the number of 0s in the first half. Taking $i = 2$ gives $0^{p+k}1^p0^p1^p$ for some $k \geq 1$, which has length $4p + k$ (odd increase), so the two halves cannot be equal.
- vxy **straddles the boundary between** 0^p **and** 1^p **(either boundary):** then pumping disrupts the pattern at that boundary. The resulting string cannot have the form ww because the symbol distribution becomes asymmetric between the two halves.
- vxy **lies within one of the** 1^p **blocks:** symmetric argument to the first case.

In every case, $uv^2xy^2z \notin D$, contradicting condition 1. Therefore D is not context-free. □

Remark 1.1: Comparing the Two Pumping Lemmas

The key differences between the two Pumping Lemmas are:

- **Regular version:** splits the string into three pieces $s = xyz$ with one pumpable section y.
- **Context-free version:** splits the string into five pieces $s = uvxyz$ with two pumpable sections v and y that must be pumped together.

This reflects the deeper structure of CFLs: where regular languages involve simple loops in a DFA, context-free languages involve nested structures in a parse tree, which naturally produce *pairs* of pumpable sections on either side of a central portion.

Note also that $\{ww\}$ is neither regular nor context-free, while $\{0^n 1^n\}$ is context-free but not regular. This illustrates the strict hierarchy: regular languages $\subsetneq$ context-free languages $\subsetneq$ all languages.

1.7 Chomsky Normal Form

When working with CFGs, it is often convenient to have them in a simplified form. One of the most useful forms is called **Chomsky Normal Form** (CNF).

Definition 1.2: Chomsky Normal Form

A context-free grammar is in **Chomsky Normal Form** if every rule is of one of the following forms:

- $A \to BC$, where A, B, C are variables and B, C are not the start variable
- $A \to a$, where a is a terminal
- $S \to \varepsilon$, where S is the start variable

The last rule is the only one permitted to have ε on the right side, and the start variable may not appear on the right side of any rule.

Theorem 1.2: Any context-free language is generated by a grammar in CNF.

Proof. We convert any CFG G into an equivalent grammar in CNF through several stages, replacing rules that violate the conditions with equivalent ones.

1. **New start variable:** Add a new start variable S_0 and the rule $S_0 \to S$, where S is the old start variable. This ensures the start variable does not appear on the right side of any rule.
2. **Eliminate ε-rules:** Remove all rules of the form $A \to \varepsilon$ (where A is not the start variable). For each rule that has A on its right side, add a new rule with A deleted. For example, if we have $B \to uAv$, we add $B \to uv$. Repeat until no ε-rules remain (except possibly $S_0 \to \varepsilon$).
3. **Eliminate unit rules:** Remove all rules of the form $A \to B$ (a single variable on the right). For each rule $B \to w$, add $A \to w$ (unless this is a unit rule previously removed). Repeat until no unit rules remain.
4. **Convert remaining rules:** Replace any rule $A \to u_1 u_2 \cdots u_k$ with $k \geq 3$ by a chain of rules, each with exactly two symbols on the right:

$$A \to u_1 A_1, \quad A_1 \to u_2 A_2, \quad \ldots, \quad A_{k-2} \to u_{k-1} u_k$$

   where $A_1, \ldots, A_{k-2}$ are new variables. If any u_i is a terminal a, replace it with a new variable U_a and add the rule $U_a \to a$.

At each stage, the grammar generates the same language. After all stages, every rule is in Chomsky Normal Form. □

2 Closure Properties of Context-Free Languages

Just as we studied closure properties of regular languages, we now examine which operations preserve context-freeness. The situation is more nuanced than for regular languages: CFLs are closed under some

operations but *not* others.

2.1 Closure Under Union

Theorem 2.1: CFLs are closed under union.

Proof. Let L_1 and L_2 be context-free languages generated by CFGs $G_1 = (V_1, \Sigma, R_1, S_1)$ and $G_2 = (V_2, \Sigma, R_2, S_2)$ respectively. We assume without loss of generality that $V_1 \cap V_2 = \emptyset$ (if not, rename the variables in one grammar — this does not change the generated language, since variables are just placeholders used during derivations).

Construct a new CFG $G = (V, \Sigma, R, S)$ where:

- $V = V_1 \cup V_2 \cup \{S\}$, where S is a new start variable
- $R = R_1 \cup R_2 \cup \{S \to S_1 \mid S_2\}$

We prove that $L(G) = L_1 \cup L_2$:

- ($\subseteq$) If $w \in L(G)$, then $S \Rightarrow^* w$. The first step must be $S \Rightarrow S_1$ or $S \Rightarrow S_2$. In the first case, the remaining derivation uses only rules from R_1, so $w \in L_1$. Similarly, in the second case $w \in L_2$.
- ($\supseteq$) If $w \in L_1$, then $S_1 \Rightarrow^* w$ using rules in R_1. So $S \Rightarrow S_1 \Rightarrow^* w$, giving $w \in L(G)$. The case $w \in L_2$ is symmetric.

Therefore $L(G) = L_1 \cup L_2$, which is context-free. □

2.2 Closure Under Concatenation

Theorem 2.2: CFLs are closed under concatenation.

Proof. Let L_1 and L_2 be context-free languages generated by CFGs $G_1 = (V_1, \Sigma, R_1, S_1)$ and $G_2 = (V_2, \Sigma, R_2, S_2)$ with $V_1 \cap V_2 = \emptyset$.

Construct a new CFG $G = (V, \Sigma, R, S)$ where:

- $V = V_1 \cup V_2 \cup \{S\}$, where S is a new start variable
- $R = R_1 \cup R_2 \cup \{S \to S_1 S_2\}$

The rule $S \to S_1 S_2$ generates a string from L_1 followed by a string from L_2. Any derivation first expands S_1 using rules from R_1 to produce some $w_1 \in L_1$, and independently expands S_2 using rules from R_2 to produce some $w_2 \in L_2$. The result is $w_1 w_2 \in L_1 \circ L_2$. Therefore $L(G) = L_1 \circ L_2$, which is context-free. □

2.3 Closure Under Kleene Star

Theorem 2.3: CFLs are closed under Kleene star.

Proof. Let L_1 be a context-free language generated by CFG $G_1 = (V_1, \Sigma, R_1, S_1)$.

Construct a new CFG $G = (V, \Sigma, R, S)$ where:

- $V = V_1 \cup \{S\}$, where S is a new start variable
- $R = R_1 \cup \{S \to SS_1 \mid \varepsilon\}$

The rule $S \to SS_1$ allows us to concatenate arbitrarily many strings from L_1, and $S \to \varepsilon$ provides the base case (the empty string). A derivation producing $w_1 w_2 \cdots w_k$ with each $w_i \in L_1$ proceeds as:

$$S \Rightarrow SS_1 \Rightarrow SS_1S_1 \Rightarrow \cdots \Rightarrow S\underbrace{S_1S_1\cdots S_1}_{k} \Rightarrow \underbrace{S_1S_1\cdots S_1}_{k} \Rightarrow^* w_1w_2\cdots w_k$$

Therefore $L(G) = L_1^*$, which is context-free. $\square$

2.4 Closure Under Intersection with Regular Languages

Theorem 2.4: CFLs are closed under intersection with regular languages.

Proof. Let A be a context-free language recognized by a PDA $P = (Q_P, \Sigma, \Gamma, \delta_P, q_P, F_P)$ and let B be a regular language recognized by a DFA $D = (Q_D, \Sigma, \delta_D, q_D, F_D)$.

We construct a new PDA M that simulates P and D simultaneously. The state set of M is $Q_P \times Q_D$, and on each input symbol M updates both the PDA component (according to δ_P, including stack operations) and the DFA component (according to δ_D). The new PDA accepts if and only if both components are in accepting states: the accept states are $F_P \times F_D$.

Since M is a PDA, it recognizes a context-free language, and by construction $L(M) = A \cap B$. Therefore $A \cap B$ is context-free. $\square$

2.5 Non-Closure Properties

Unlike regular languages, CFLs are *not* closed under intersection or complement. These results have important consequences.

CFLs Are Not Closed Under Intersection

Theorem 2.5: CFLs are not closed under intersection.

Proof. We exhibit a counterexample. Consider the languages:

$$A = \{a^n b^n c^m \mid n, m \geq 0\} \qquad \text{and} \qquad B = \{a^m b^n c^n \mid n, m \geq 0\}$$

Both A and B are context-free. For A, the grammar

$$S \to S_1 S_2, \quad S_1 \to aS_1 b \mid \varepsilon, \quad S_2 \to cS_2 \mid \varepsilon$$

generates $a^n b^n c^m$ by matching the as and bs independently of the cs. Similarly, B has a grammar that matches bs and cs independently of the as:

$$S \to S_1 S_2, \quad S_1 \to aS_1 \mid \varepsilon, \quad S_2 \to bS_2 c \mid \varepsilon$$

However, their intersection is:

$$A \cap B = \{a^n b^n c^n \mid n \geq 0\}$$

We proved earlier (using the Pumping Lemma for CFLs) that this language is not context-free. Therefore CFLs are not closed under intersection. $\square$

CFLs Are Not Closed Under Complement

Theorem 2.6: CFLs are not closed under complement.

Proof. We prove this by contradiction using De Morgan's law. Suppose CFLs were closed under complement. Since CFLs are closed under union, for any context-free languages A and B we would have:

$$A \cap B = \overline{\overline{A} \cup \overline{B}}$$

The right side would be context-free: $\overline{A}$ and $\overline{B}$ would be context-free (by assumed closure under complement), their union would be context-free (by closure under union), and the complement of their union would be context-free (by assumed closure under complement).

But this would mean CFLs are closed under intersection, which we just proved is false. Contradiction. Therefore CFLs are not closed under complement. $\square$

2.6 Summary of Closure Properties

Remark 2.1: Comparison of Closure Properties

Operation	**Regular Languages**	**CFLs**
Union	✓	✓
Concatenation	✓	✓
Kleene Star	✓	✓
Intersection	✓	✗
Complement	✓	✗
Intersection with Regular	✓	✓

The fact that CFLs are closed under intersection with regular languages but not under intersection with other CFLs is particularly noteworthy. This is a useful tool: if A is context-free and R is regular, then $A \cap R$ is guaranteed to be context-free. This can be used to prove languages are *not* context-free by contradiction (similar to how we used closure of regular languages to give an alternative proof that $C = \{w \mid w \text{ has equal 0s and 1s}\}$ is nonregular).